Lab 3: Local search

Authors: Piotr Foltyniewicz 160295, Jakub Adamski 160291

Problem description

Each instance consists of 200 nodes. A row in the input file contains the node's planar coordinates and selection cost, in the form `x;y;cost`. Node indices are zero-based and correspond to row order. The task is to select exactly 100 distinct nodes (half of the available nodes), order them, and form a closed cycle that minimizes the sum of travel distances and selected-node costs.

Distances are Euclidean, rounded to the nearest integer and precomputed after reading the instance. All algorithms use this distance matrix. The evaluation sums all selected-node costs and all cycle edges, including last -> first. In the pseudocode, `m = 100`, `n = 200`, `c[v]` is node cost, and `d(a, b)` is the precomputed distance. A solution `S` is an ordered list of `m` distinct nodes; `U` denotes the set of nodes not in `S`.

The objective for an ordered selection `S` is:

```
F(S) = sum(c[S[i]]) + sum(d(S[i], S[(i + 1) mod length(S)]))
```

Pseudocode

All indices are zero-based. Local search starts from a given solution and applies moves until none improves the objective. Moves are evaluated only through deltas; the neighbor solution is never built and `F` is never recalculated from scratch. The eight methods combine: steepest or greedy search, node or edge exchange as the intra-route move, random or greedy starting solution.

Neighborhood and deltas

The neighborhood consists of inter-route moves and one kind of intra-route moves. A move is only a descriptor (type and positions).

- `INTER(p, v)`: replace the selected node at position `p` with an unselected node `v`.
- `SWAP_NODES(i, j)`, `i < j`: exchange the nodes at positions `i` and `j`.
- `SWAP_EDGES(i, j)`, `i < j`: replace edges `(S[i], S[i+1])` and `(S[j], S[j+1])` with `(S[i], S[j])` and `(S[i+1], S[j+1])` by reversing `S[i+1..j]`. Edges sharing a node are skipped.

Positions are taken modulo `m`, so the closing edge is handled like the others.

```
DELTA_INTER(S, p, v):
    out = S[p]; prev = S[p - 1]; next = S[p + 1]
    return d(prev, v) + d(v, next) - d(prev, out) - d(out, next) + c[v] - c[out]

DELTA_SWAP_EDGES(S, i, j):
    a = S[i]; b = S[i + 1]; e = S[j]; f = S[j + 1]
    return d(a, e) + d(b, f) - d(a, b) - d(e, f)
```

```
DELTA_SWAP_NODES(S, i, j):
    E = distinct edge starts among {i - 1, i, j - 1, j}
    return sum over e in E of d'(e, e + 1) - d(e, e + 1)     // d' = edge length
after the swap
```

Steepest local search

```
STEEPEST(S):
    loop:
        best = NONE; bestDelta = 0
        for move in all moves:
            delta = DELTA(S, move)
            if delta < bestDelta: best = move; bestDelta = delta
        if best == NONE: return S
        APPLY(S, best)
```

Greedy local search

Randomization. All moves of both kinds (inter-route and the chosen intra-route kind) form one set of move descriptors. Moves are taken from it in uniformly random order: each time a move is needed, a random not yet examined move is drawn, so the two kinds are mixed and the search does not start preferentially from either of them. Deltas are not precomputed: only the drawn move is evaluated, and the first move with a negative delta is applied immediately. Therefore only as many moves are evaluated as are needed to find an improvement (all of them only at a local optimum). After a move is applied, the set of moves is generated again. The random generator is `std::mt19937` seeded from `std::random_device`.

```
GREEDY(S):
    loop:
        R = all moves                                  // both kinds
        improved = false
        while R is not empty:
            move = random element removed from R   // uniformly random order
            if DELTA(S, move) < 0:
                APPLY(S, move)
                improved = true
                break
        if not improved:
            return S
```

Starting solutions

Random: the first `m` nodes of a random permutation of all `n` nodes. Greedy: the best construction heuristic of labs 1-2 for the instance, started from each of the 200 nodes: weighted-sum greedy cycle for TSPA and weighted-sum nearest neighbor for TSPB.

Each method was run 200 times per instance (200 random solutions, or each node as the construction start). Times include the construction of the starting solution.

Results

Each method was tested 200 times on TSPA and TSPB. Lower objective values are better. Results of labs 1 and 2 are copied from the previous reports. Methods of this lab are in bold. Tables are sorted by the mean objective; times are averages per run. For lab 3, "Starting solution" is the average time of generating the starting solution (the greedy construction heuristic, or a random permutation) and "Local search" is the remainder of the run (Avg Time minus Starting solution).

TSPA

Method	Runs	Minimum	Maximum	Mean	Avg Time (ms)	Starting solution (ms)	Local search (ms)
Greedy Edges Greedy	**200**	**71955**	**74861**	**72984.00**	**1.922548**	**1.414950**	**0.507598**
Steepest Edges Greedy	**200**	**71955**	**74861**	**73003.80**	**1.743490**	**1.414950**	**0.328540**
Greedy Nodes Greedy	**200**	**71955**	**74877**	**73020.10**	**2.043134**	**1.414950**	**0.628184**
Steepest Nodes Greedy	**200**	**71955**	**74861**	**73021.71**	**1.967720**	**1.414950**	**0.552770**
Greedy Cycle heuristics with a weighted sum criterion	200	72011	75532	73353.51	5.075577	–	–
Greedy Cycle	200	72590	75281	73743.73	3.582711	–	–
Nearest Node Any	200	72328	77022	74306.53	2.884855	–	–
NN heuristics with a weighted sum criterion	200	72419	78504	75561.20	5.377398	–	–

Method	Runs	Minimum	Maximum	Mean	Avg Time (ms)	Starting solution (ms)	Local search (ms)
Greedy Edges Random	**200**	**74104**	**79090**	**76212.29**	**7.242936**	**0.003531**	**7.239405**
Steepest Edges Random	**200**	**73478**	**80392**	**76261.07**	**10.064306**	**0.003531**	**10.060775**
Nearest Node End	200	84362	90075	87476.63	0.060209	–	–
Greedy Nodes Random	**200**	**81592**	**95442**	**88483.99**	**8.018548**	**0.003531**	**8.015017**
Steepest Nodes Random	**200**	**83531**	**97983**	**89934.21**	**19.707589**	**0.003531**	**19.704058**
Greedy Cycle 2-regret heuristics	200	112806	128089	120378.76	5.229371	–	–
NN heuristics 2-regret heuristics	200	110950	135771	121360.29	5.459556	–	–
Random	200	233146	288073	260752.93	0.007843	–	–

TSPB

Method	Runs	Minimum	Maximum	Mean	Avg Time (ms)	Starting solution (ms)	Local search (ms)
Steepest Edges Random	**200**	**50020**	**55780**	**52544.65**	**10.120907**	**0.003519**	**10.117388**
Greedy Edges Random	**200**	**50306**	**55983**	**52966.58**	**7.276858**	**0.003519**	**7.273339**
Steepest Edges Greedy	**200**	**49346**	**60354**	**53091.28**	**2.548794**	**1.411800**	**1.136994**

Method	Runs	Minimum	Maximum	Mean	Avg Time (ms)	Starting solution (ms)	Local search (ms)
Greedy Edges Greedy	**200**	**49692**	**60357**	**53172.22**	**2.607454**	**1.411800**	**1.195654**
Steepest Nodes Greedy	**200**	**49978**	**61265**	**54292.07**	**2.864336**	**1.411800**	**1.452536**
Greedy Nodes Greedy	**200**	**49996**	**61265**	**54294.05**	**2.473184**	**1.411800**	**1.061384**
NN heuristics with a weighted sum criterion	200	50485	61406	55698.34	5.332107	–	–
Greedy Cycle heuristics with a weighted sum criterion	200	52731	63370	57866.33	5.140883	–	–
Nearest Node End	200	56007	63166	59397.33	0.086600	–	–
Nearest Node Any	200	52915	66164	59476.22	4.494223	–	–
Greedy Cycle	200	55655	63734	59650.81	3.416733	–	–
Greedy Nodes Random	**200**	**58230**	**69865**	**63898.63**	**7.870025**	**0.003519**	**7.866506**
Steepest Nodes Random	**200**	**59309**	**74904**	**66211.37**	**19.354679**	**0.003519**	**19.351160**
Greedy Cycle 2-regret heuristics	200	68150	77274	72484.84	5.307665	–	–
NN heuristics 2-regret heuristics	200	68761	79756	73649.84	5.374936	–	–
Random	200	179244	236859	206352.15	0.010232	–	–

Note: labs 1-2 were run on a different machine than lab 3, so their times are not directly comparable with the times of lab 3.

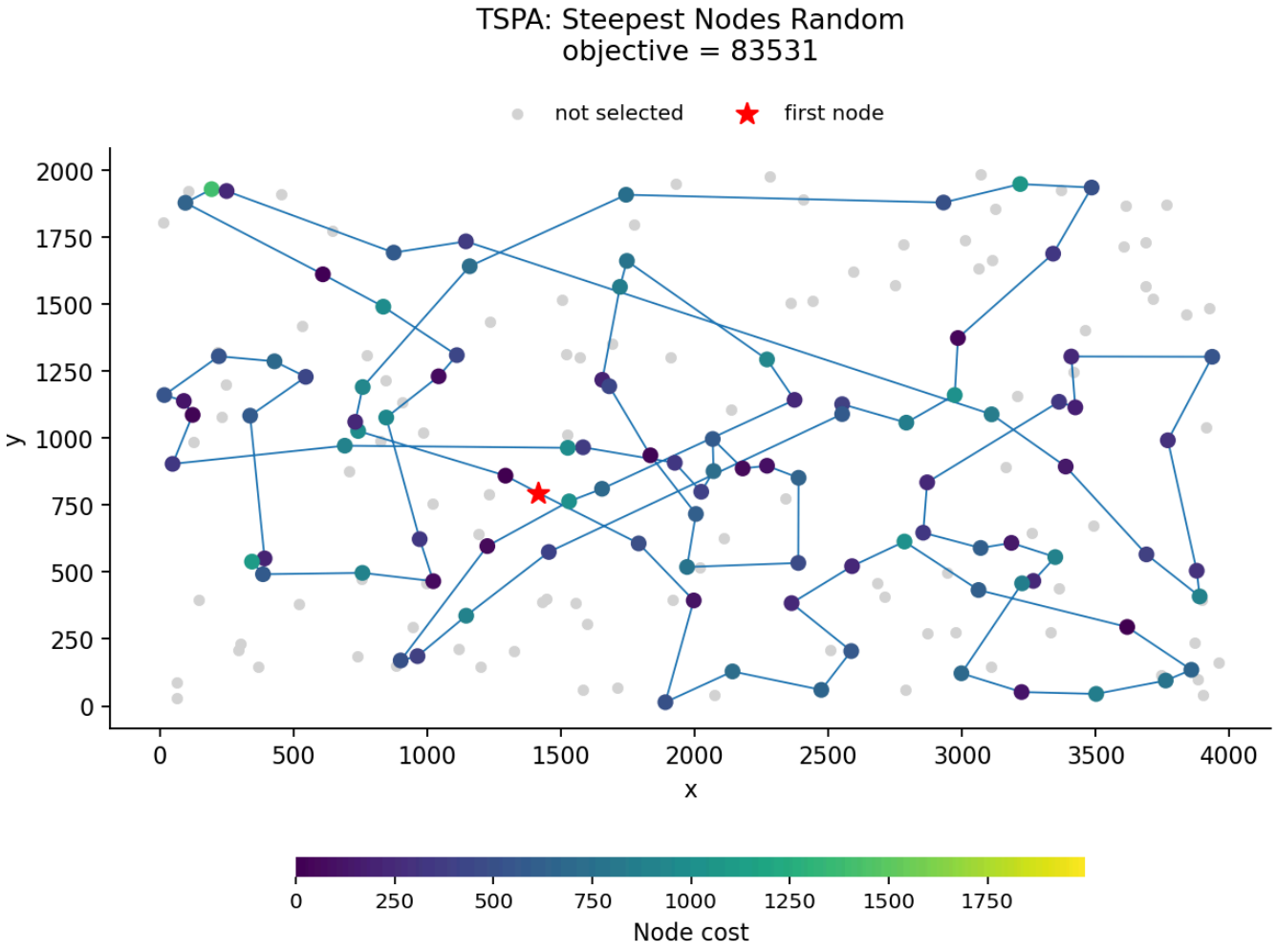
Best solutions and visualizations

Node color represents selection cost; the compact horizontal scale below each plot uses the same cost range for all methods on an instance. Gray nodes are not selected, blue segments form the closed tour, and the red star marks the first node in the displayed sequence. Lists below show ten node indices per row; read left to right, then top to bottom to follow the tour.

TSPA

Steepest Nodes Random

Objective: **83531**.



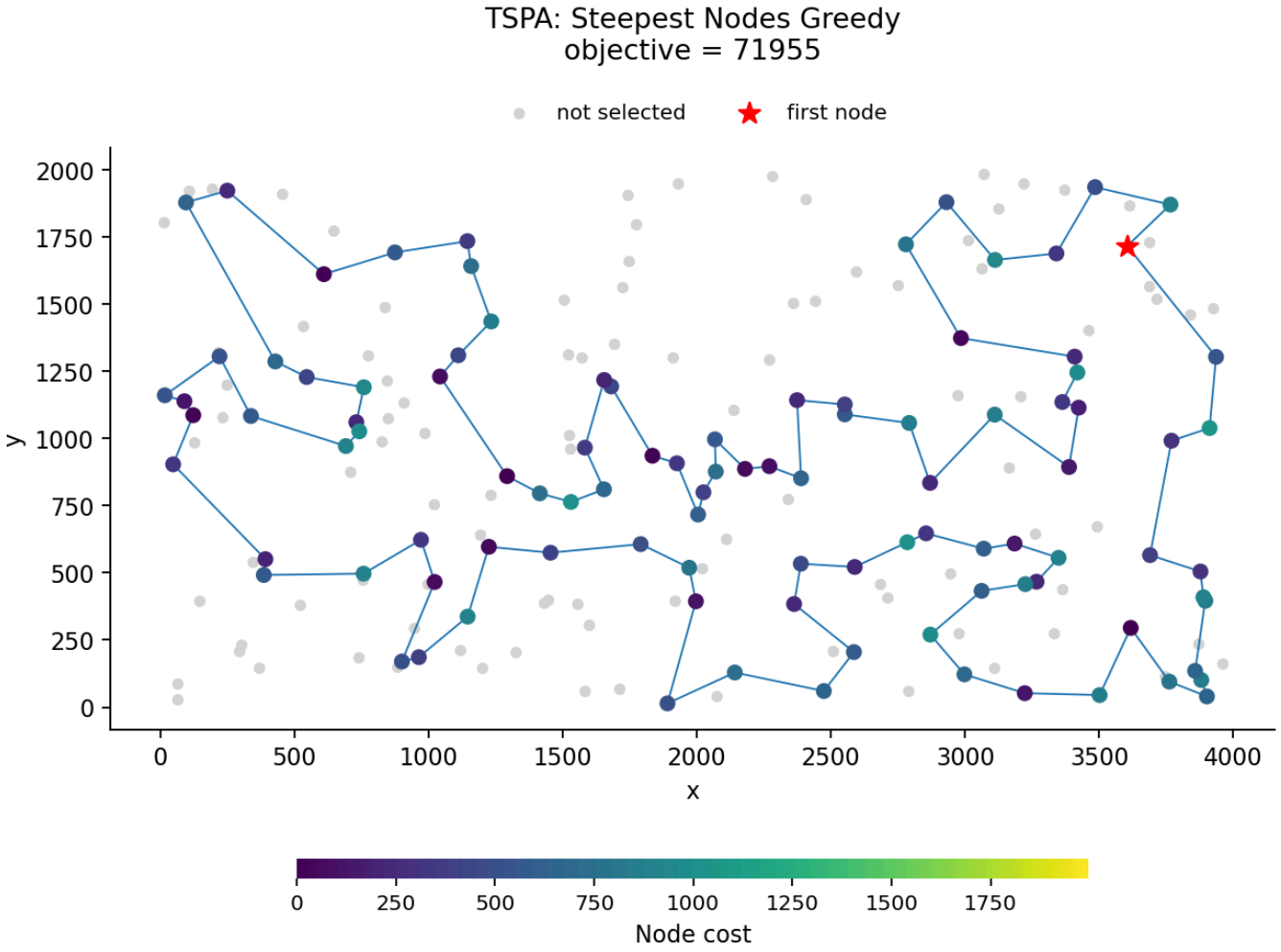
Solution (tour order):

```
 28 180  83 123 100  71 115 136 163  29
178 164 142   8   9  54  87 189   2 171
141 102 185  33 143 112 139  51  16  67
140  73  84 131  36 177  52 179  53 144
 35 156 160 168  63 149 116  60 108 152
 49  97  95 127 161 187  30  90 132  76
```

```
 38  80  31   7  48  91 157  85  93 130
146 159  47 147   1  50   3 174 176 150
106 113  89  42 182  57  64  88 153 107
151  72 134  58 158 192 181 114 135  45
```

Steepest Nodes Greedy

Objective: **71955**.

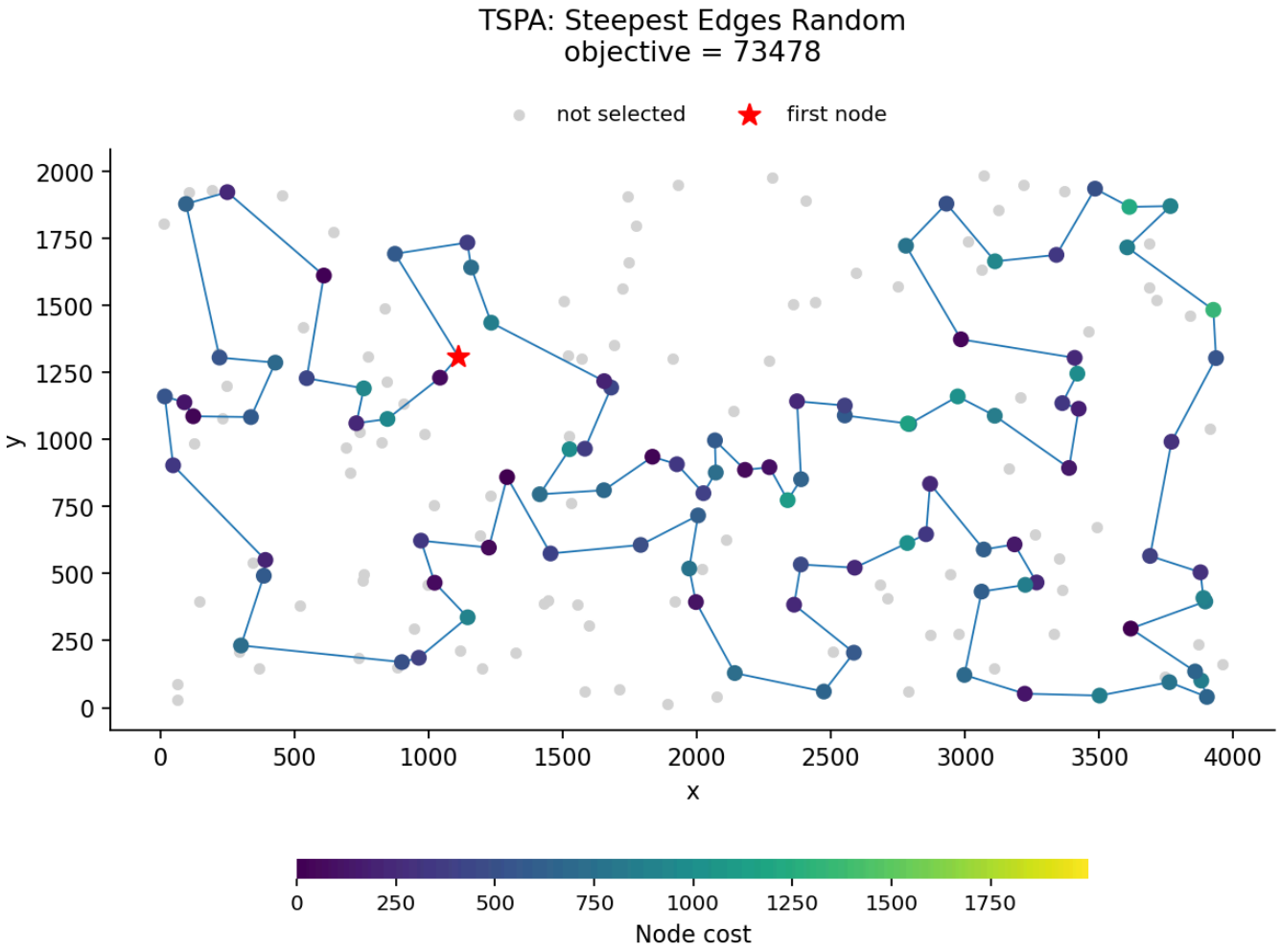


Solution (tour order):

```
101  55  29 178 184 136 186 164   1 169
  3  50  93  85 174   8  54   9  33 131
 36 177  52 179  53 140 144  67  16  51
 35 185 102  28 180 132  76   4  71 157
 91  80  48  31 108 152 100 123  83 160
 49  60 116 149  63 168  97 127 161  30
187 171   2 189 141  87  45  73 135 114
181 192 158  58  84 134  72 176 150 106
113  89  42 151 138 182  57  64 107  88
 78 193 153 170 146 159 130  47  37 147
```

Steepest Edges Random

Objective: **73478**.

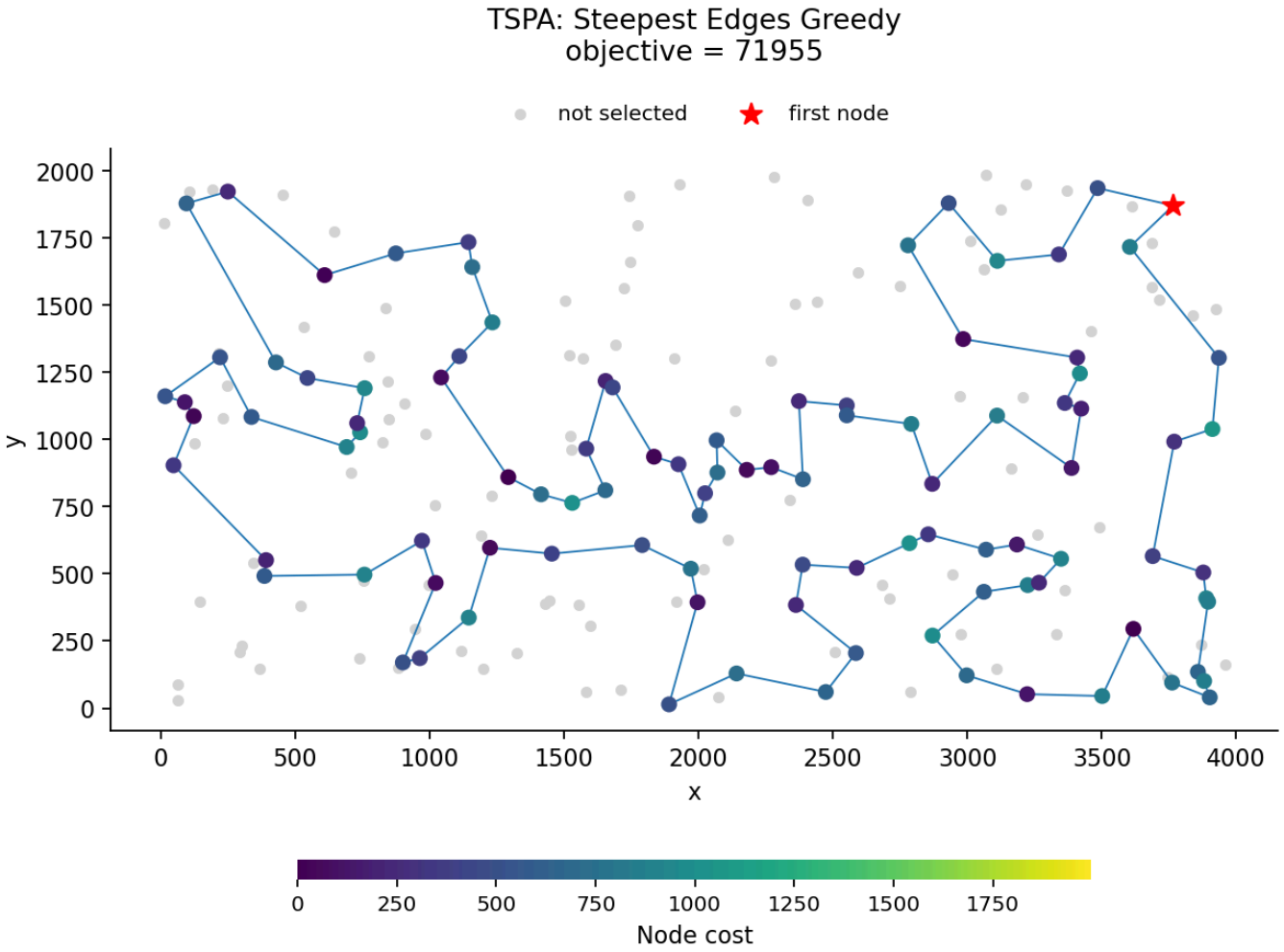


Solution (tour order):

```
 76 132  90 123 100 152  80  48  31  60
108  49  63 149 116 168  97 127  69 171
  2 189 187  30 141 180  87  45 140  73
135 181 192 158  58  84 134  72 176 174
150 106  89  42 151 182  57  64  88  78
193 153 107 170 146 159 130  47 147  21
101  55  17  29 178 184 136 186 164   1
169   3  50  93  85 142   8 194  54   9
 33 131  99  36 177  52 179  53 144  67
185  28 156  35  16  51   4  71 157  91
```

Steepest Edges Greedy

Objective: **71955**.

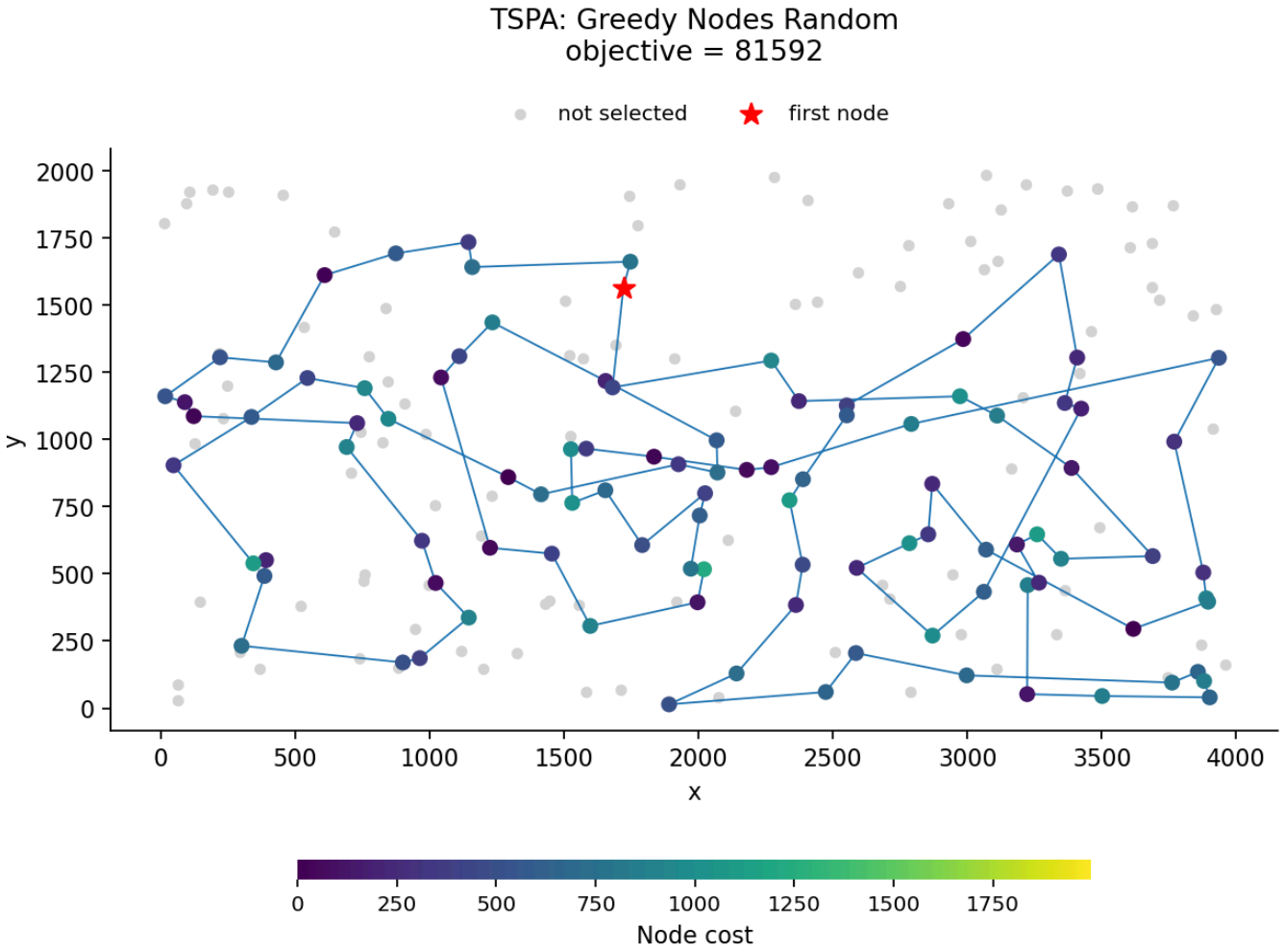


Solution (tour order):

```
 55 101 147  37  47 130 159 146 170 153
193  78  88 107  64  57 182 138 151  42
 89 113 106 150 176  72 134  84  58 158
192 181 114 135  73  45  87 141 189   2
171 187  30 161 127  97 168  63 149 116
 60  49 160  83 123 100 152 108  31  48
 80  91 157  71   4  76 132 180  28 102
185  35  51  16  67 144 140  53 179  52
177  36 131  33   9  54   8 174  85  93
 50   3 169   1 164 186 136 184 178  29
```

Greedy Nodes Random

Objective: **81592**.

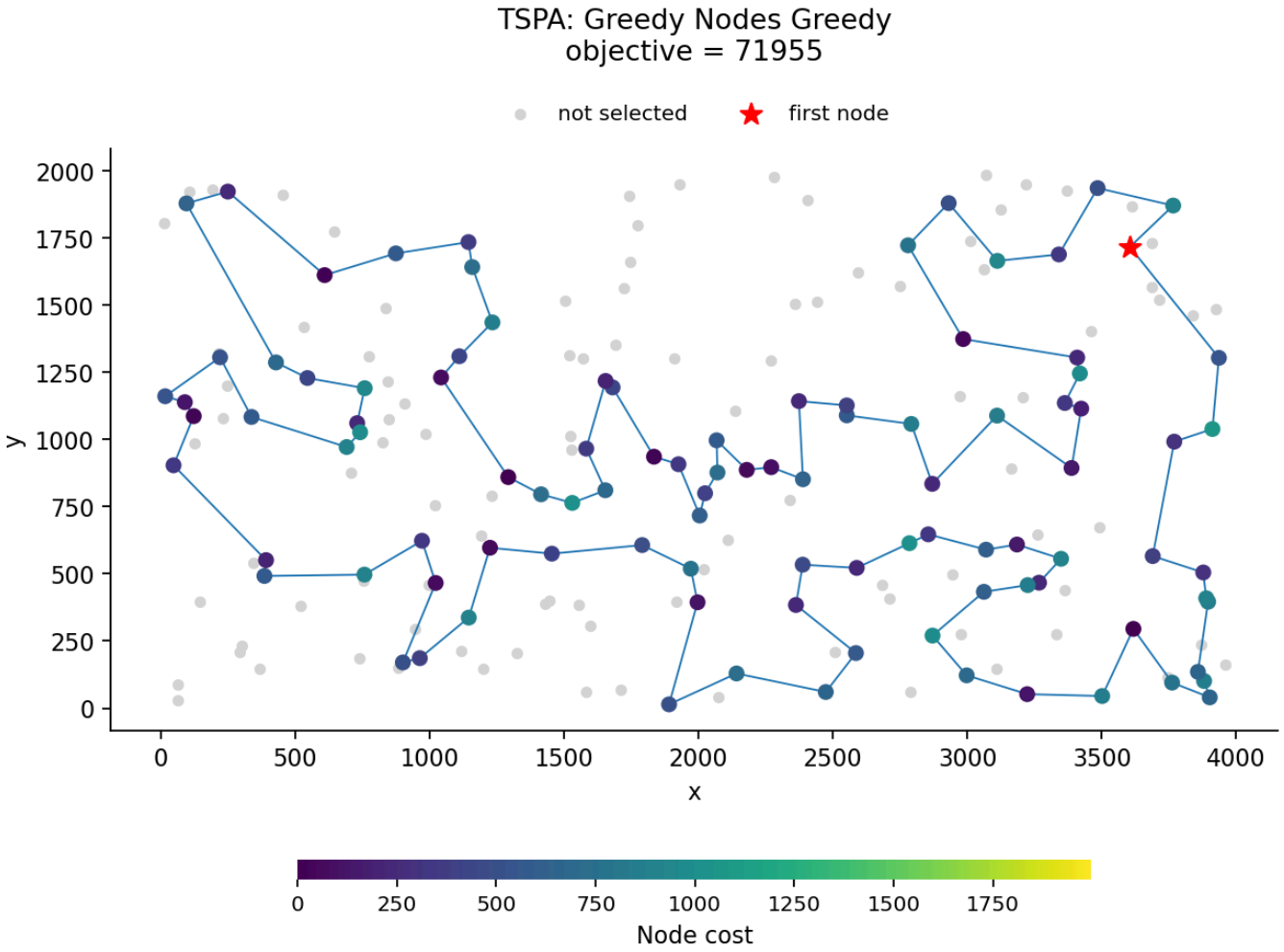


Solution (tour order):

```
139 112  71 157  91  80 108  60 116 149
 63 123 160  30 187 189   2 171  69 127
 97  95 168  49 152 100  90 180  28 144
179  52  51   4  76 132 141  87  98 135
 61  73 140  53  45 185 102 156  35  67
177  36   8 147  47 159 146 170 107 150
174 176  72 134 138 151  50   3   1 178
164   9  54 131  99  84  58 181 114 192
158 182  88 153 193  78  64  57  42  89
106  43 113 130  93  85 142  33 143  16
```

Greedy Nodes Greedy

Objective: **71955**.

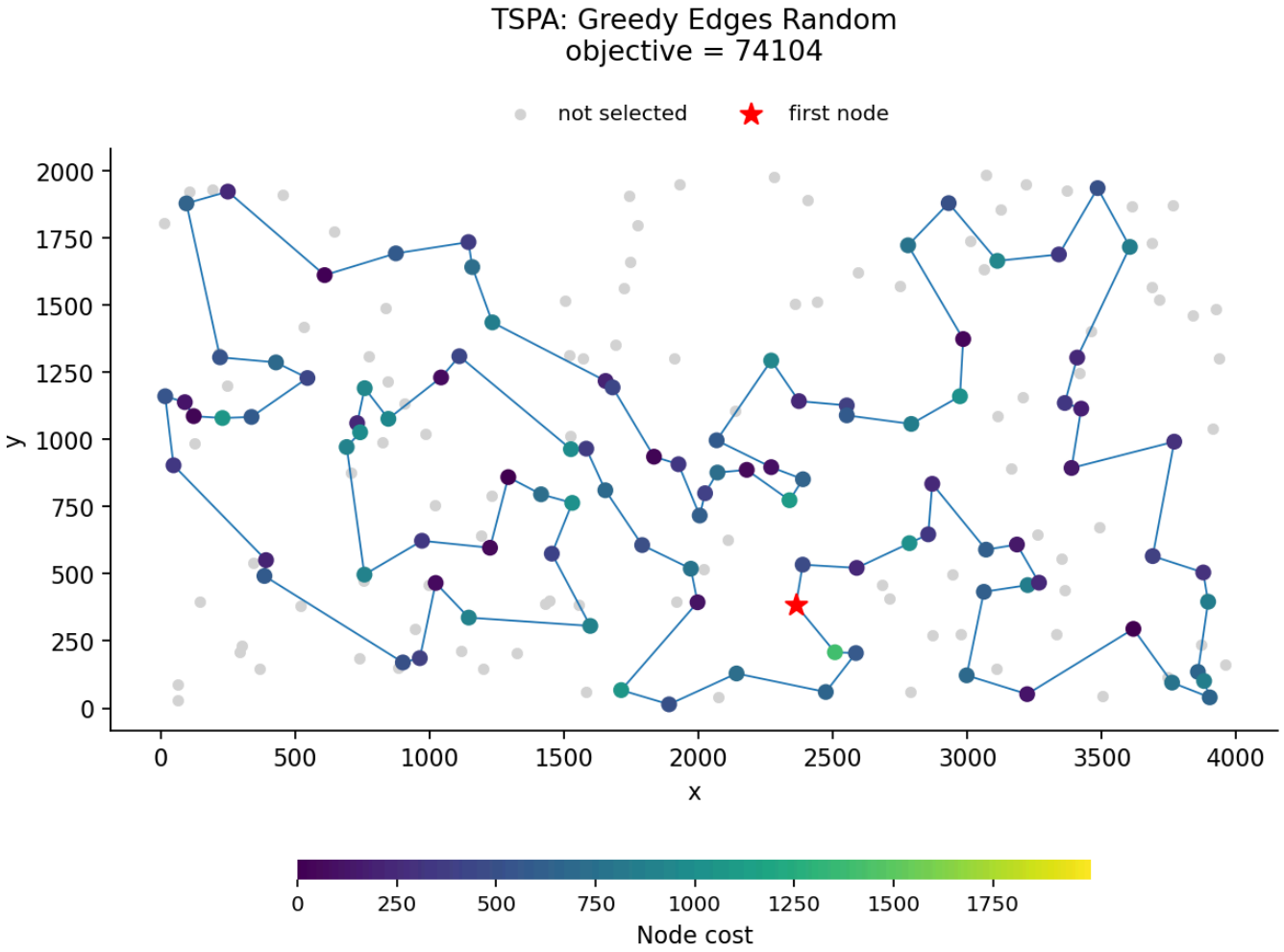


Solution (tour order):

```
101  55  29 178 184 136 186 164   1 169
  3  50  93  85 174   8  54   9  33 131
 36 177  52 179  53 140 144  67  16  51
 35 185 102  28 180 132  76   4  71 157
 91  80  48  31 108 152 100 123  83 160
 49  60 116 149  63 168  97 127 161  30
187 171   2 189 141  87  45  73 135 114
181 192 158  58  84 134  72 176 150 106
113  89  42 151 138 182  57  64 107  88
 78 193 153 170 146 159 130  47  37 147
```

Greedy Edges Random

Objective: **74104**.

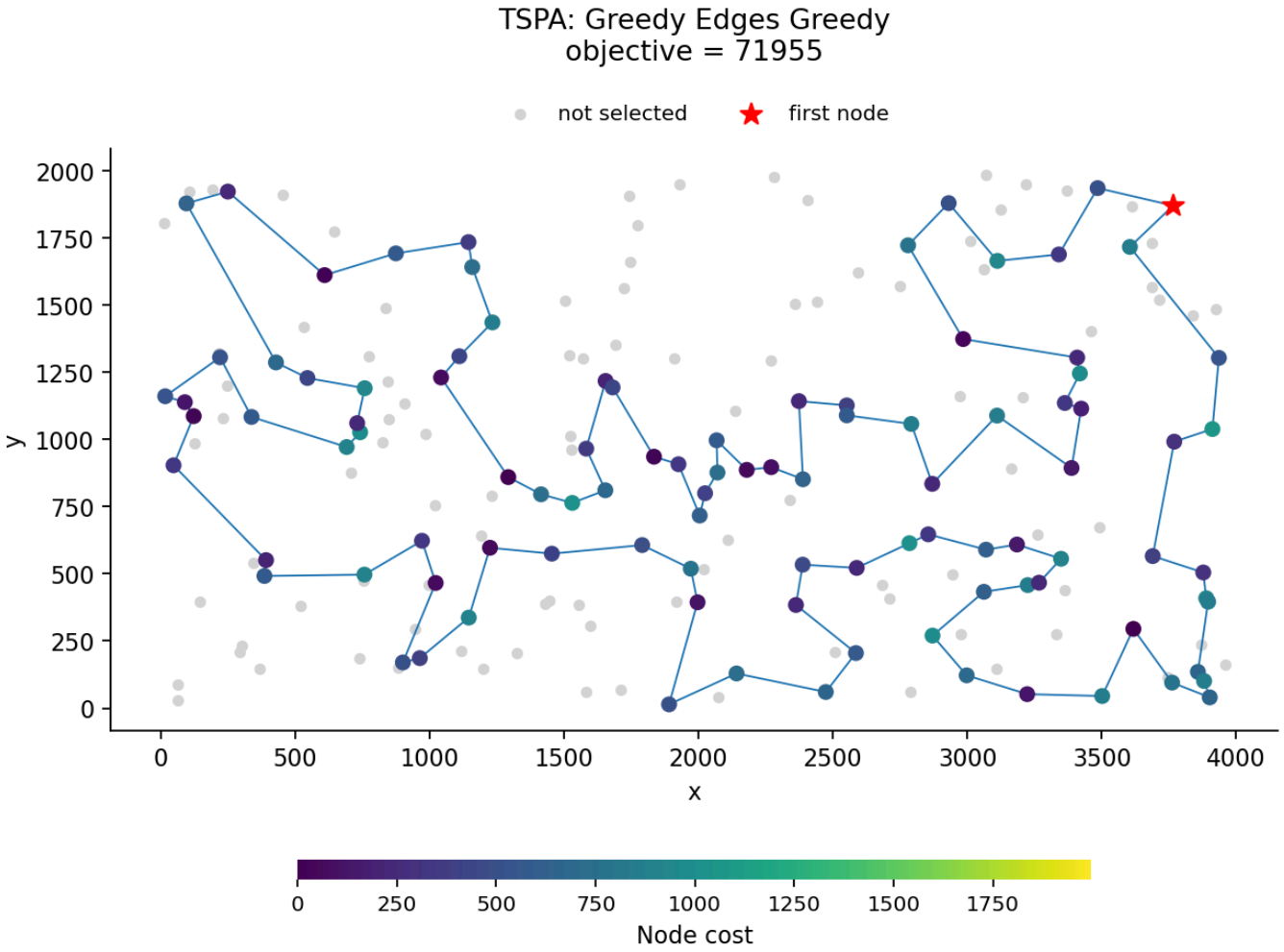


Solution (tour order):

```
 58  20 158 192 181 114  25 135  73  45
185  35 156  76 132  90 100 123  83 160
161  30 141 180  28 102  87  98 189 187
  2 171 127  97 168 116 149  63  11  49
152 108  60  31  48  80  91 157  71   4
 51  16  67 144 140  53 179 177  99 131
 36  52 143  33   9  54   8 142 164 186
136 184 178  29 101   1   3  50  93  47
130 159 170 153 193  78  88 107  57 182
151  42  89 106 150 174 176  72 134  84
```

Greedy Edges Greedy

Objective: **71955**.



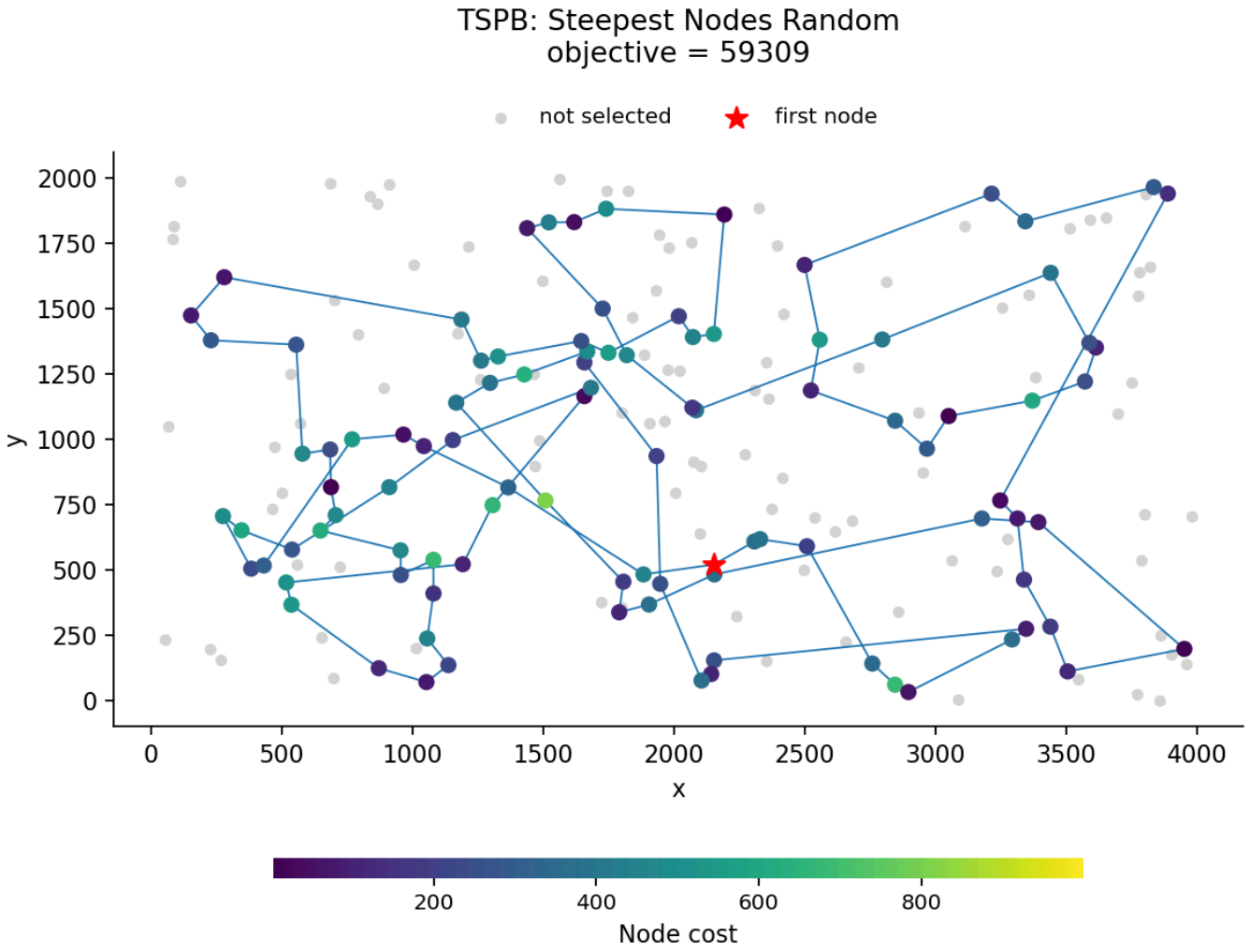
Solution (tour order):

```
 55 101 147  37  47 130 159 146 170 153
193  78  88 107  64  57 182 138 151  42
 89 113 106 150 176  72 134  84  58 158
192 181 114 135  73  45  87 141 189   2
171 187  30 161 127  97 168  63 149 116
 60  49 160  83 123 100 152 108  31  48
 80  91 157  71   4  76 132 180  28 102
185  35  51  16  67 144 140  53 179  52
177  36 131  33   9  54   8 174  85  93
 50   3 169   1 164 186 136 184 178  29
```

TSPB

Steepest Nodes Random

Objective: **59309**.

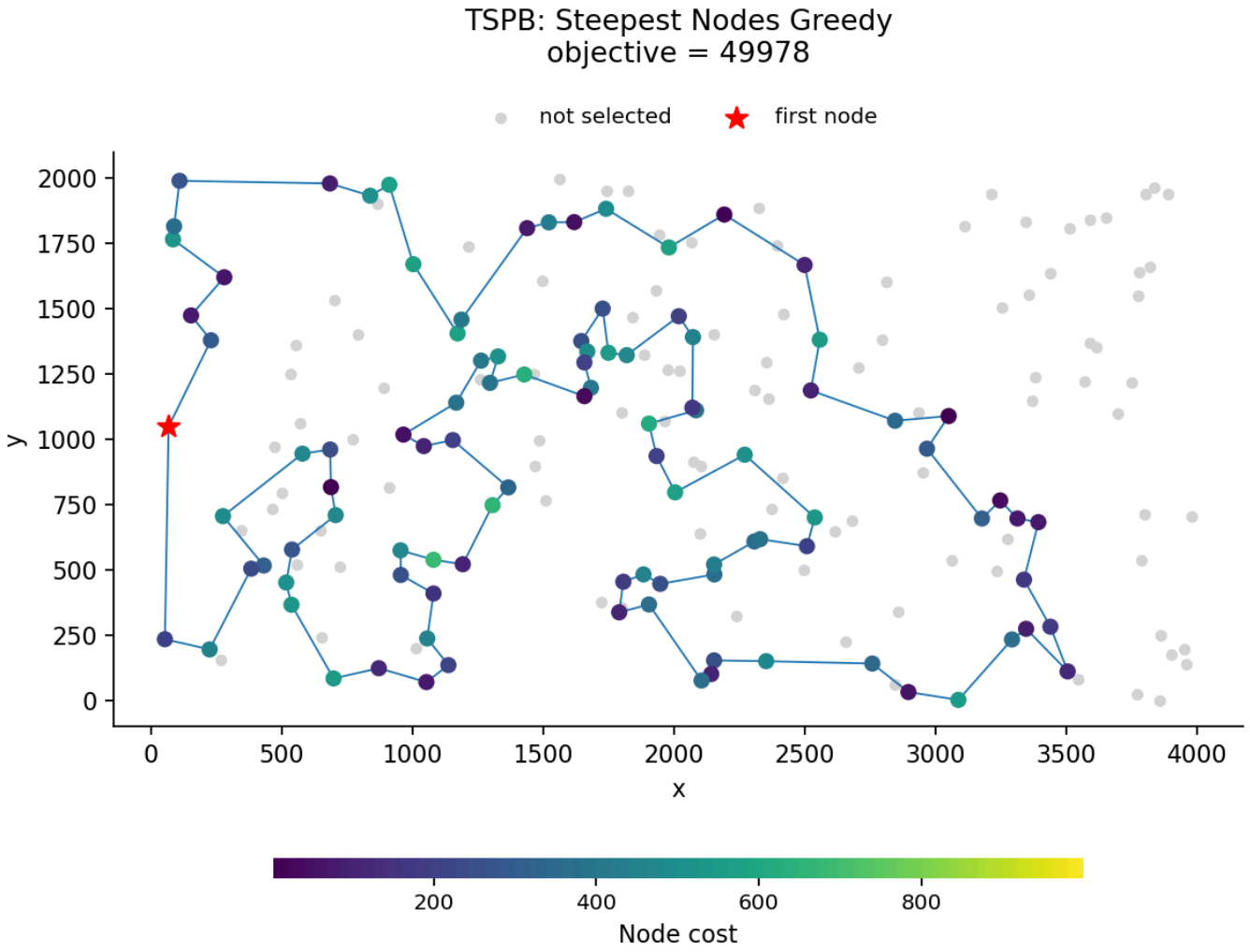


Solution (tour order):

```
152  34  24  67  99  39 171  74  20 179
 87  82 167 151 172 136 189  21  38  49
 59  85  51 181  80  90 110   2  14 153
129 127 105  94  70 190  57 185  76 162
192  54  78  10 114 196 187 104   3 184
117 120 121 198  96  48  89  86  46  72
  8 133 183  30 130 149  28 145 158 177
125 170 115 180 155  71 141  60 157  45
134 175  36 197  40  33  69 160  47  97
  6 103 178 132  29  26 154 140  64  73
```

Steepest Nodes Greedy

Objective: **49978**.

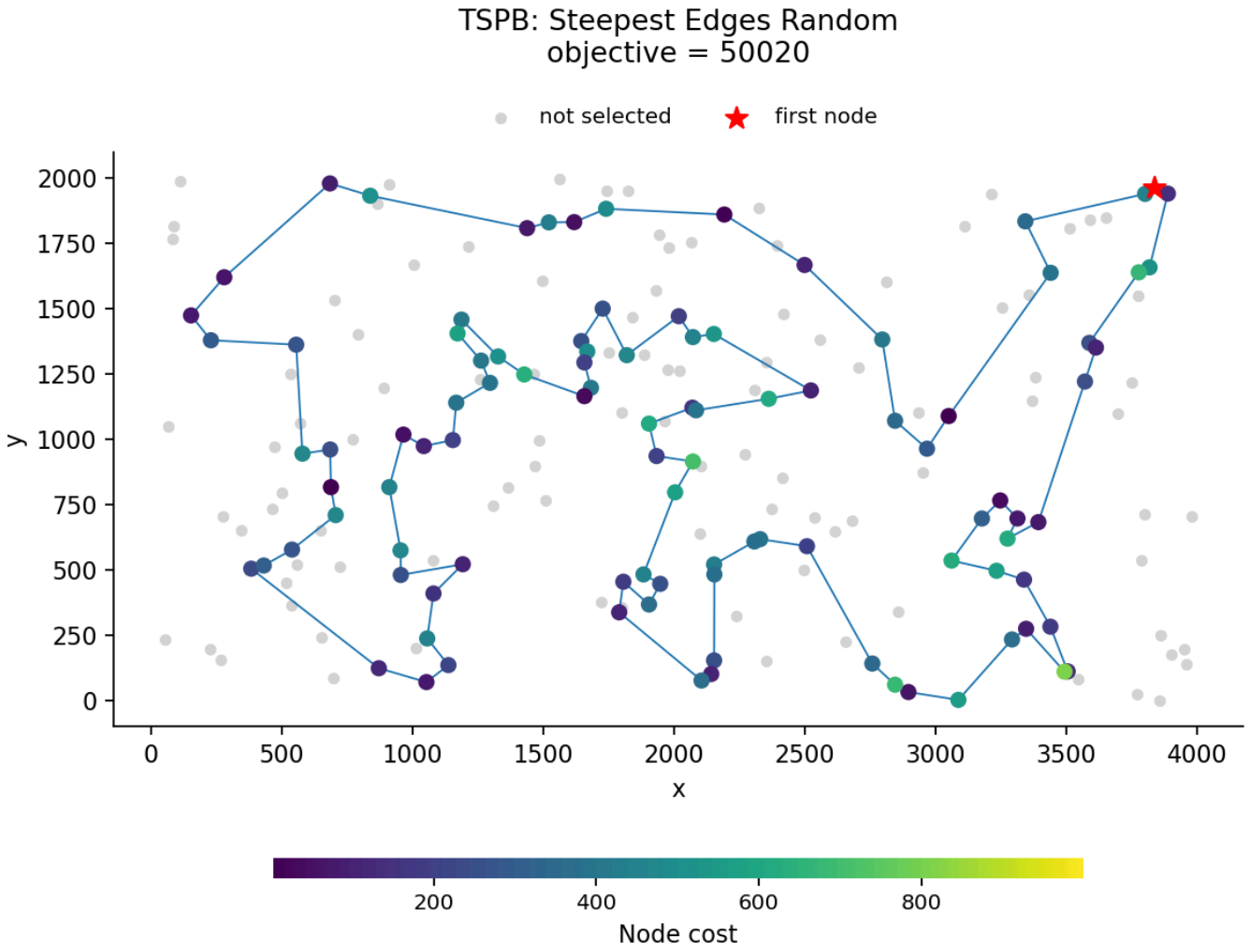


Solution (tour order):

```
 65 145  28 149  13 146  37 116 147  42
 50 191 130 121 198  96  48 156  89  57
185  76 162  54 192  80 127 129  90 153
 14   2  20  74  17 171  99  88 179  87
 82  51  85  59  73 167 181 152  34  24
 67  66  27  98 151 122   3 184  46  72
117   8 120 133 136 172 160  69 189  21
183  30  38 154 140  47  64  33  40 141
155  71  60 157  45 134 175  62  36 197
  6 115 170 125 177 178  29 132 174   1
```

Steepest Edges Random

Objective: **50020**.

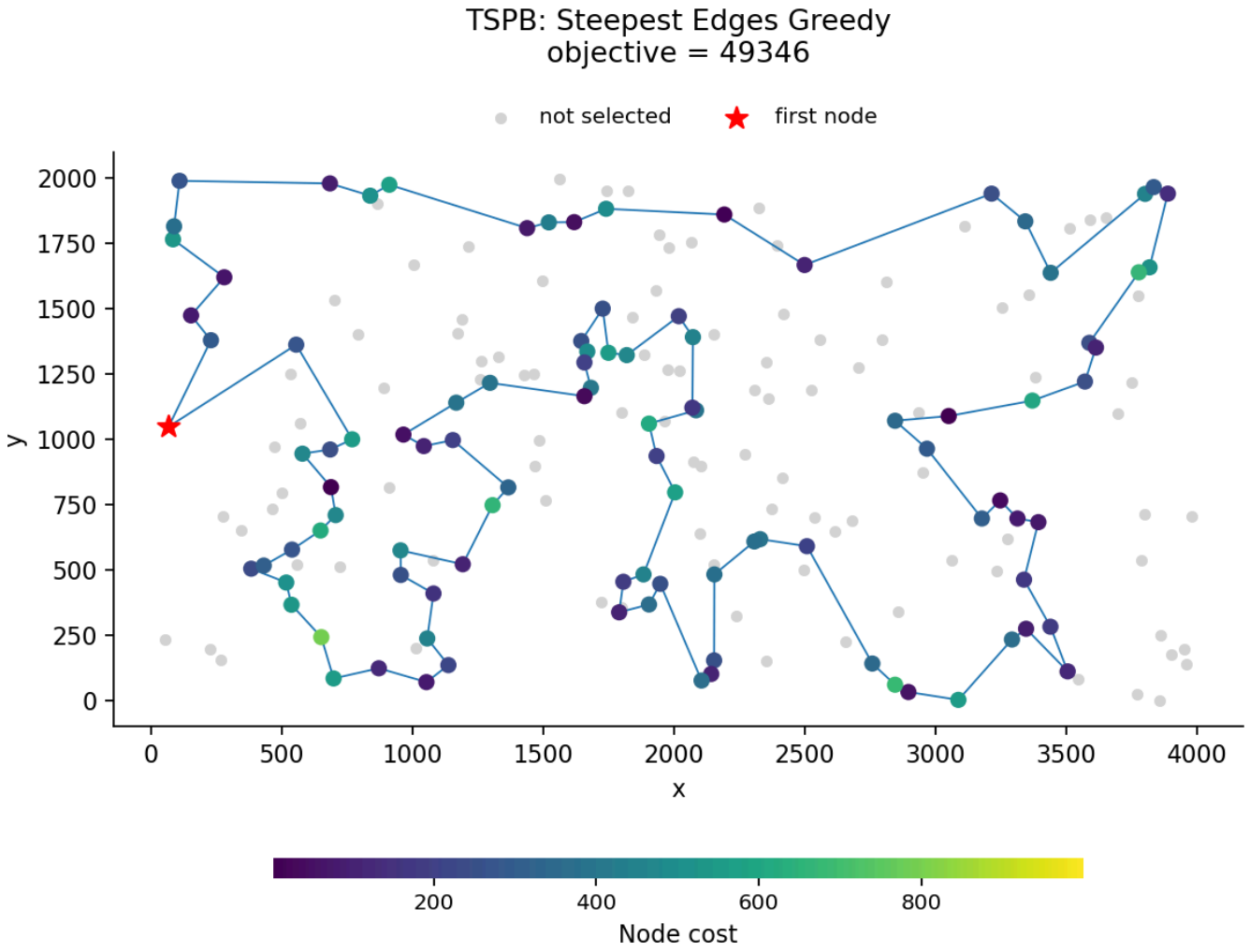


Solution (tour order):

```
 94 105  61 139 196 114  10  90  52 129
127  80 150 182 153  14   2 188  20  74
 17 171  39  99  67  24  34 152 181 179
 87  82  85  59  51 167  73  98 111 151
122 184   3 135  76  86  46  72 117 120
133 136 172 160  69 189 183 130 191  30
 21  38  47 140 154  97 155  71  40  60
157  45 134 175 132  29   6 115 170 125
177 158 145  28 149 116 147 121 198  96
 48  89  57 104 162 192  54 187  70  81
```

Steepest Edges Greedy

Objective: **49346**.

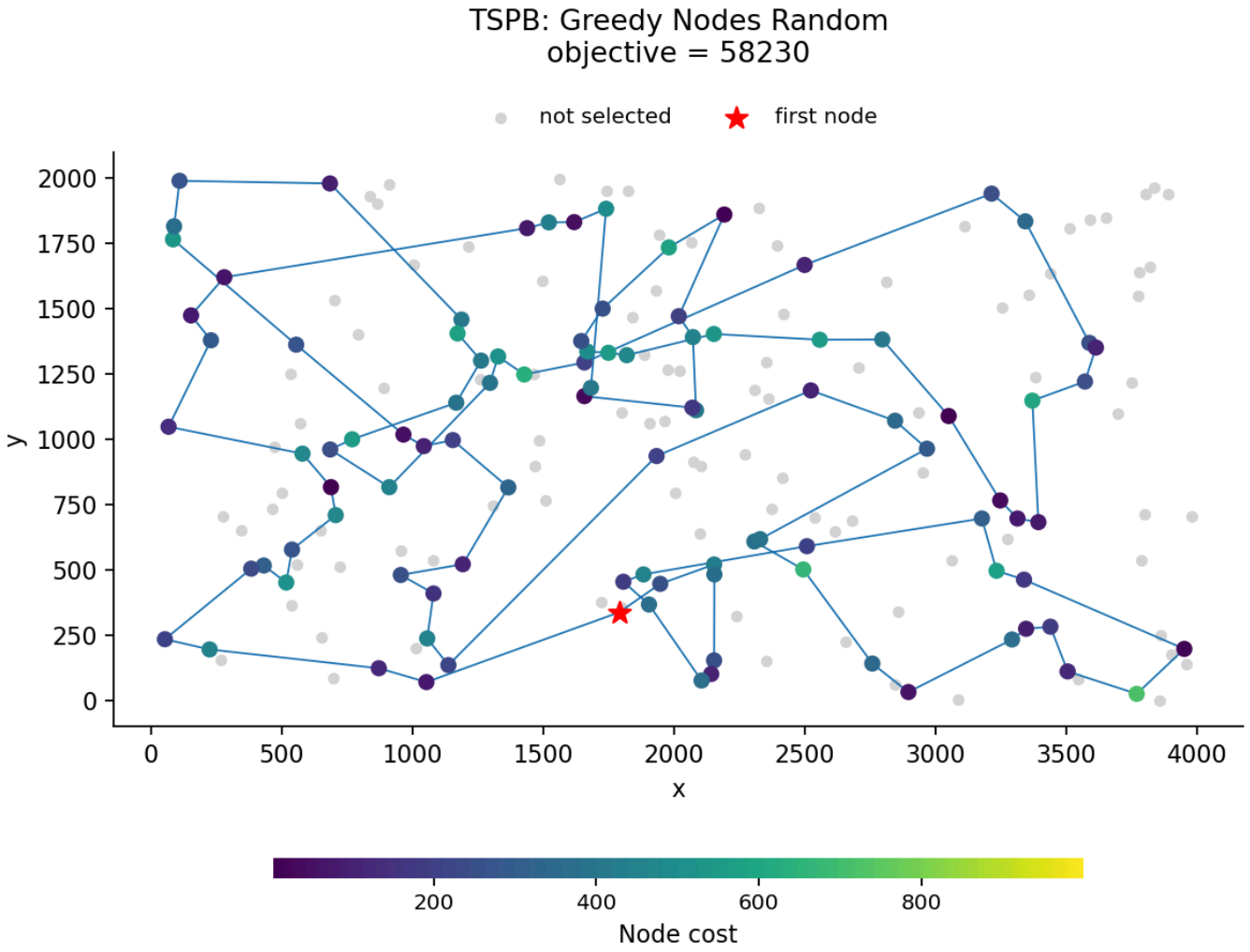


Solution (tour order):

```
  65 145  28 149  13 146  37 116 147  42
 121 198  96  48  89  57 190  70 187  81
  94 105  61 139 196 114  10  78  54 162
 192  80 127 129  90 153  14   2  20  74
  17 171  39  99  67  24  34 181 179  87
  82 167  51  85  59  73  98 151 122   3
 184  46  72 117   8 120 133 136 172 160
  69  21  38 154 140  47  64  33  40 155
  71  60 157  45 134 175  62  91  36 197
 132  29   6 180 115 170 177 125  26 158
```

Greedy Nodes Random

Objective: **58230**.

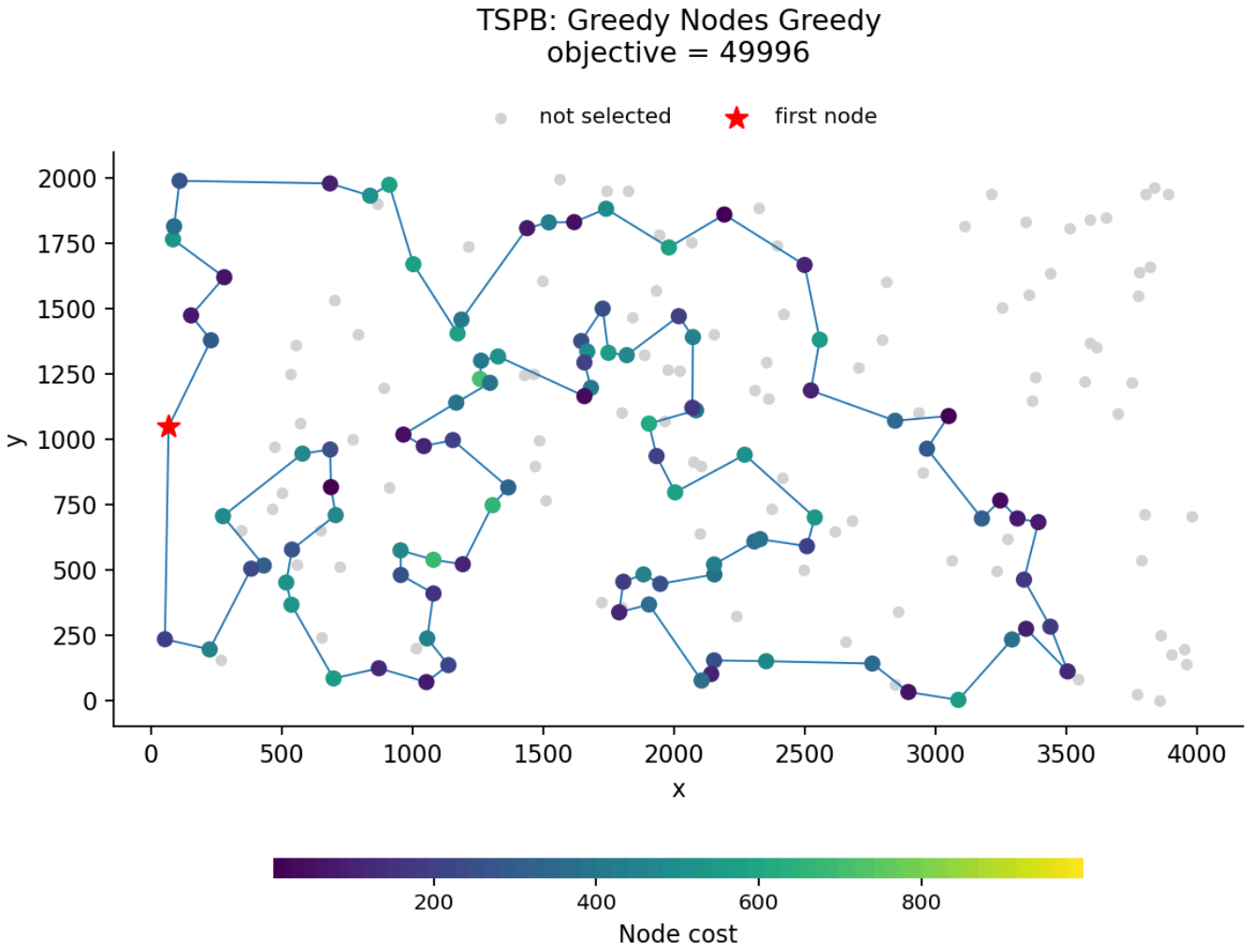


Solution (tour order):

```
 85 167 152 181 179  87  82  51  59  73
 67  80 182 153 110  93   2  14  20  74
171  99  31  34  24 192 162  76 151  45
157  60  71  40  64  47 140 154 158  13
146  37 116 130 191  30  38  26 125  97
 21 183 189 172  57 190  70 196 114  10
 78  90 129 127  54 104 185  86 117   8
136 133 120 156  89  72  46   3 184  69
160  48  96 198 121 149  28 145  65 177
170 115   6 197  29 132   1 174 175 134
```

Greedy Nodes Greedy

Objective: **49996**.

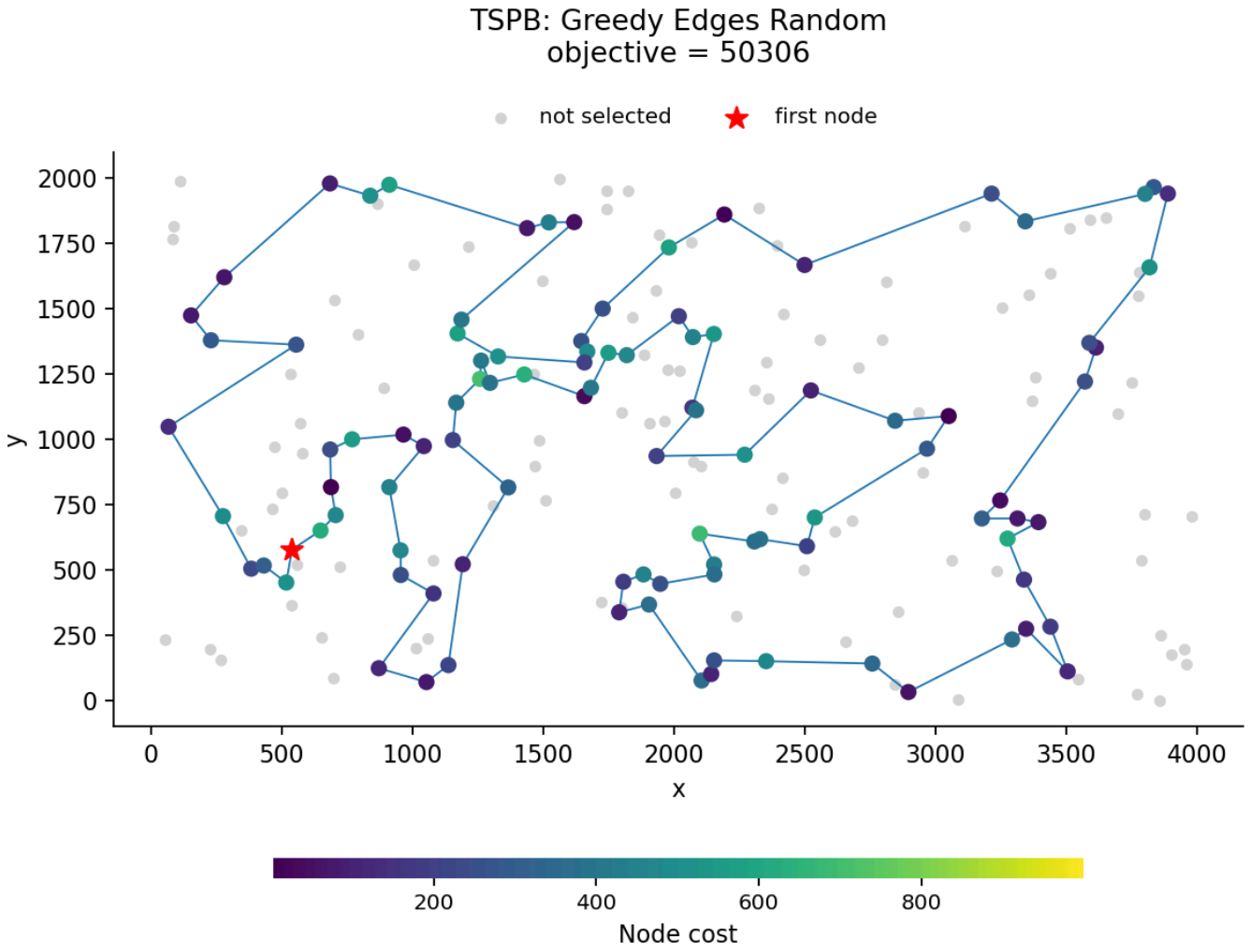


Solution (tour order):

```
 65 145  28 149  13 146  37 116 147  42
 50 191 130 121 198  96  48 156  89  57
185  76 162  54 192  80 127 129  90 153
 14   2  20  74  17 171  99  88 179  87
 82  51  85  59  73 167 181 152  34  24
 67  66  27  98 151 122   3 184  46  72
117   8 120 133 136 172 160  69 183  30
176  21  38 154 140  47  64  33  40 141
155  71  60 157  45 134 175  62  36 197
  6 115 170 125 177 178  29 132 174   1
```

Greedy Edges Random

Objective: **50306**.

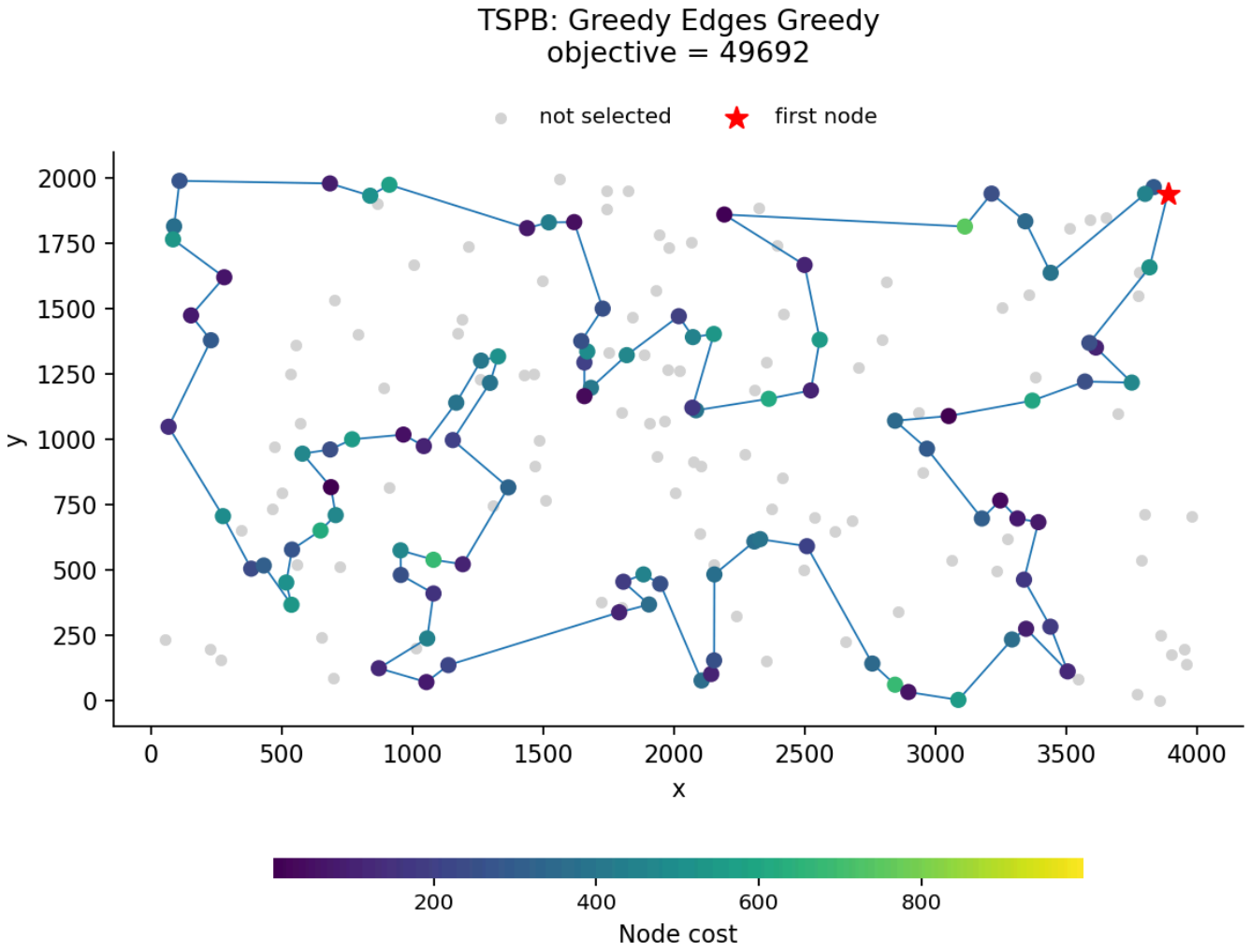


Solution (tour order):

```
  6 180 115 170 125  26 154 140  97 155
 71  60 175 134  45  40  64  47  38 176
 30  21 189  69 160   8 117  72  46  86
184   3 151  27  76 162  54 192  66  67
 24  34  25 152 181 167  73  59  85  51
 82  87 179  88  99 171  74  20   2  14
153  52  90 129  80 127  10 114 196  61
105  94  81  70 190  57  89 156 120 133
136 172 183 191 130  96 198 121  42 147
116 149  28 145 158  65 178 132  29 197
```

Greedy Edges Greedy

Objective: **49692**.



Solution (tour order):

```
105  94  81 187  70 190 194  89  57 185
 76 135   3 184  86  46  72 117 160  69
172 136 133 120  96 198 121  42 147 116
 37 146  13 149  28 145  65 178 132  29
 36 197   6 180 115 170 177 125  26 154
140  38  30 183  21  47  64  40 141 155
 71  60 157 175 134  45  85  51  59  73
167  82  87 179 181  34  24  67  99  39
171  17  74  20   2  14 153  90 129 127
 80 192 162  54  78  10 148 114 196  61
```

Solution checking

All 16 best solutions were checked for exactly 100 distinct indices in the range 0-199, and their objectives match the minima in the lab 3 summary.

Source code

Repository: PiotrFoltyniewicz/Evolutionary-Computation.

- Local search (steepest, greedy, moves, starting solutions)

- Experiment runner
- Experiment statistics and timing
- Visualization script
- Exported solutions
- Lab 3 summary

Conclusions

Local search improved the greedy starting solutions: on TSPA the best average is 72984.00 (Greedy Edges Greedy, previously 73353.51) and the best objective 71955 (previously 72011), found by all variants started from the greedy construction; on TSPB the best average is 52544.65 (Steepest Edges Random) and the best objective 49346 (Steepest Edges Greedy, previously 50485). The gain in the average over the greedy starting solutions is about 0.5% on TSPA and 2.5-4.7% on TSPB.

Edge exchange is clearly better than node exchange, especially from random starts (TSPA 76.2-76.3k vs 88.5-89.9k, TSPB 52.5-53.0k vs 63.9-66.2k). From greedy starts the difference is negligible on TSPA and about 1.1-1.2k on TSPB. From greedy starts steepest and greedy search give similar quality (differences below 0.2%). From random starts greedy search is better with node exchange (by 1.6-3.5%), while with edge exchange the two are equal on TSPA and steepest is better by 0.8% on TSPB.

Greedy search is faster than steepest from random starts: about 7.2-8.0 ms vs 10.1 ms (edges) and 19.4-19.7 ms (nodes), because it evaluates only the moves up to the first improvement. From greedy starts few iterations are needed and the times are similar (1.7-2.9 ms in total, of which the construction takes about 1.4 ms). On TSPB, random starts with edge exchange have a better average and a smaller spread than greedy starts (maximum 55.8-56.0k vs 60.4k), although the best solutions come from greedy starts.